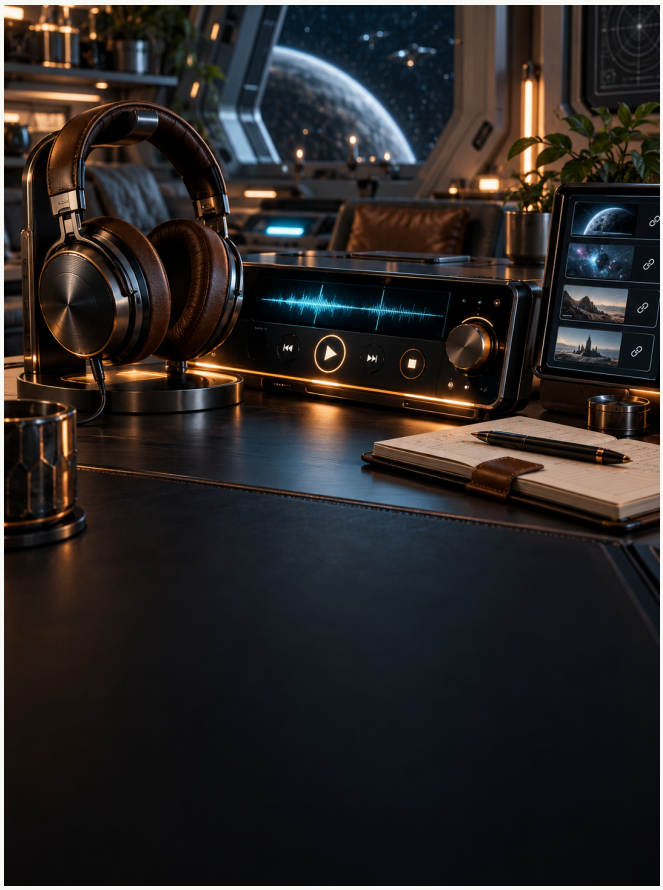
Music Reference Hub

A learner needs to listen to local audio, keep reference links, and record what helped during a study or creative session.

Open the raw template. Add one part. Run it. Explain it. Save a Git checkpoint.



HTML tutorial: music-quick-start.html

Each chapter names the file, the code to add, its meaning, and a test. The raw starter contains TODO comments, not a finished utility.

Your chapter route

01 / Begin with a user and a need - 00 / PRE-START

02 / Open your raw WPF project - 00 / PRE-START

03 / Read the files before adding code - 8A / UNDERSTAND

04 / Build and understand the interface - 8A / VISUAL HAT

05 / Create your first GitHub checkpoint - 8A / SAVE THE FOUNDATION

06 / Give the window data it can remember - 8A / DATA HAT

07 / Load and play your music - 8A / FIRST WORKING ACTION

08 / Keep and open useful reference links - 8B / ADVANCE

09 / Export and reopen a readable record - 8B / SAVE AND RECOVER

10 / Add artwork and an image button - 8B / VISUAL HAT

11 / Test, ask AI, and resolve a real unknown - 8B / VERIFY

12 / Package the app and submit the GitHub link - WEEK 8 / FINAL

13 / Complete code reference - RECOVERY / AFTER THE CHAPTERS

Day 1: Chapters 1-7. Day 2: Chapters 8-11. Weekly release: Chapter 12. Chapter 13 is a recovery reference. Canvas owns official deadlines.

Action 8A / Plan and build the first working action

Tutorial chapters 1-7

Whiteboard one flow, explain it, build it, and check it with a peer. This list summarizes the existing work; it adds no requirements.

Context and Attention First

Identify the user, need and assumptions; apply SEEK and AAA to the first action.

Prototype and flowchart

Sketch the screen and trace input -> event -> method -> data -> output.

Class, object and scope

Locate MainWindow, its constructor, fields and methods; explain where each belongs.

XAML and events

Use layout, properties and named controls; connect a button Click to its C# handler.

Variables and types

Use text and true/false values; distinguish a field from a local variable.

Collections and binding

Compare an array with ObservableCollection; connect the collection through ItemsSource.

Methods and the first action

Call a focused method from an event handler and check its visible result.

Git and GitHub

Save, commit and push the starter and first working checkpoint; explain the difference.

Audio playback

Choose a local audio file, set MediaElement.Source and use Play, Pause and Stop.

Day recap: Explain one button from its XAML declaration to the C# method, changed data and visible result.

Action 8B / Complete, test and revise the same app

Tutorial chapters 8-11; final handoff in chapter 12

Whiteboard one flow, explain it, build it, and check it with a peer. This list summarizes the existing work; it adds no requirements.

Validation and recovery

Use conditions, early return and try/catch to handle mistakes with useful feedback.

Parameters and return values

Pass information into a method and return a result; explain void versus a returned value.

Text-file input and output

Use file dialogs, WriteAllText and ReadAllText to export and reopen a text record.

Images and readable UI

Load a BitmapImage, place an image inside a labelled button, and check contrast.

Testing, AI and peer review

Test normal, invalid and recovery routes; test one AI suggestion and record the decision.

Revision and handoff

Push the revised GitHub checkpoint; reuse class evidence in README, PATTERN.md and the receipt.

Playback state and reference links

Handle media events and readiness; validate URLs and open a selected link in the browser.

Day recap: Demonstrate a mistake and recovery, reopen an exported text file, and explain one reusable pattern.

Begin with a user and a need

A learner needs to listen to local audio, keep reference links, and record what helped during a study or creative session.

1. Choose a course music track, give your hub a cover image, save a research link, and write one observation about the listening session.

2. Write Context.md: who uses this, when, what they need, what you assume, and what you must test. Use the Nexus setting, your own story, or a real client.

3. Apply the assigned Attention First SEEK and AAA work: make the first action visible, give feedback, and provide a way to recover.

4. Whiteboard the story -> user -> input -> event -> method -> data -> result. Plan the screen in PowerPoint or a photo editor before XAML.

Understand the concepts

Context

The situation that gives the app a purpose. The technical features serve that purpose.

Assumption

A claim you have not tested yet, such as “my user will recognize this button.” Test it with a peer.

RUN AND CHECK

Keep the brief, prototype, and whiteboard photo for Action 8A.

Open your raw WPF project

Windows Presentation Foundation (WPF) builds Windows desktop interfaces. Visual Studio manages the project, XAML, C#, debugger, and Git.

1. Download only raw-music-reference-hub.zip. Right-click it -> Extract All into Documents/CS077/Week8/MusicReferenceHub.

2. Use Visual Studio 2022 or later with the .NET 8 SDK and .NET desktop development workload. Select Open a project or solution -> MusicReferenceHub.csproj. This starter targets .NET 8; keep its target framework.

3. In Solution Explorer, find MainWindow.xaml and expand it to find MainWindow.xaml.cs. Press F5 to run. Click a button: the status line should say its behavior is still to build.

4. If you choose to start from a blank project, select C# WPF Application (.NET), name it exactly as this guide, and use a matching .NET framework. Then use the raw starter XAML and placeholder C# below.

5. Create Assets/Images, Assets/Media, and Evidence folders in the project. Keep class notes and test files in Evidence. Stop debugging before editing.

Understand the concepts

.csproj

The project recipe: framework, compiler settings, and WPF support.

Solution

A workspace that can hold one or several projects.

Assets

Images and media used by your application.

Build vs run

Build checks and compiles code; run starts the application so you can test its behavior.

RUN AND CHECK

F5 opens the raw window. No finished list, timer, or player behavior is expected yet.

Read the files before adding code

The window is a class. XAML describes its controls; the partial C# class supplies its behavior. Braces keep code in the correct scope.

1. Open MainWindow.xaml. x:Class names the namespace and class. x:Name gives a control a name that C# can use. Click names the handler to run.

2. Open MainWindow.xaml.cs. using lines belong at the top, above namespace. Fields belong inside MainWindow, outside methods. Helper methods and event handlers also belong inside MainWindow, alongside the constructor.

3. The constructor runs when the window is created. InitializeComponent() builds the controls from XAML; call it before using any named control.

4. A method is a named unit of work, like a box with instructions inside its braces. Call it to run those instructions. A parameter brings a value into it. A return value sends a result back.

Understand the concepts

Class

A definition of a kind of object, with its data and actions. MainWindow inherits the behavior of a WPF Window.

Object

A particular instance made from a class. Your running window is an object.

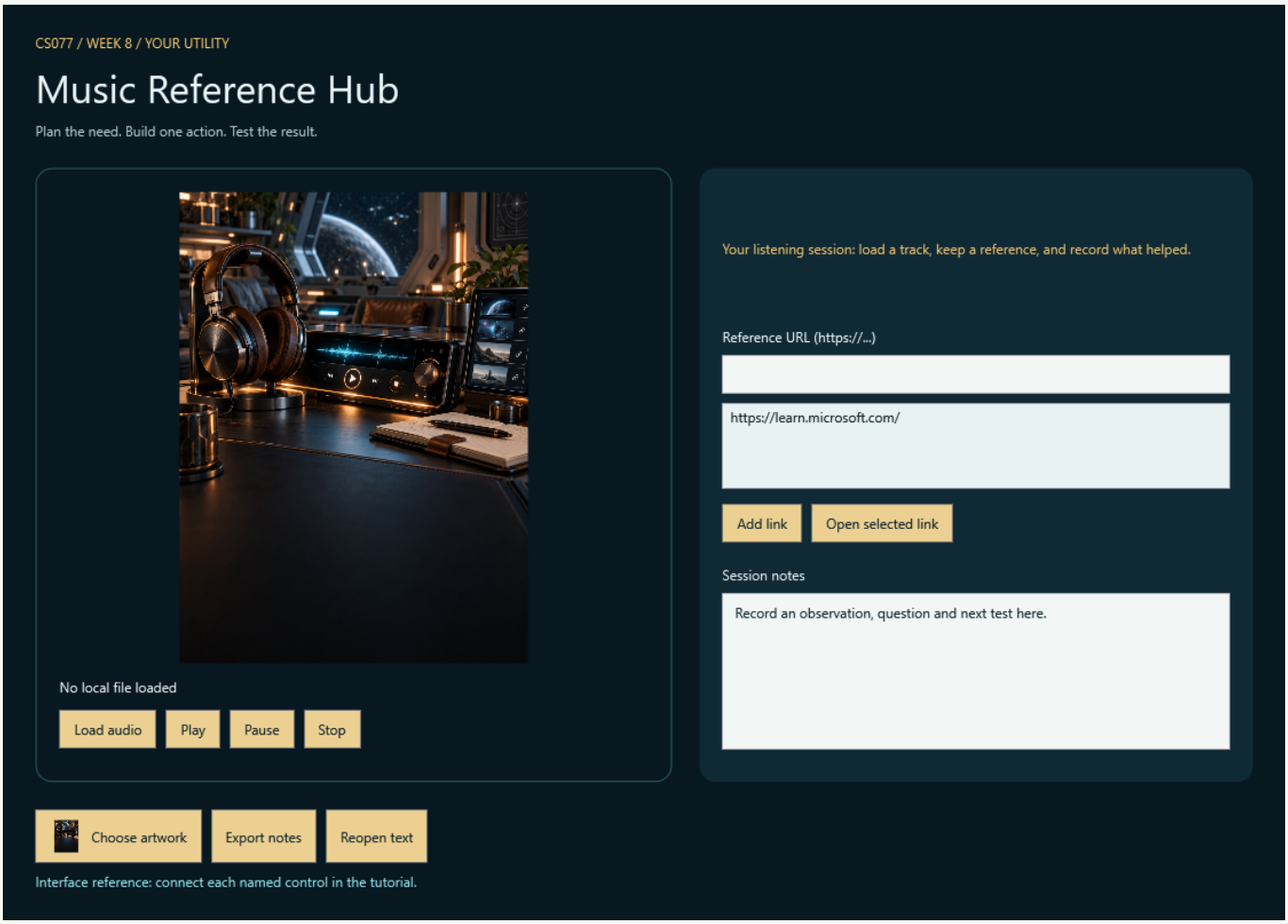
partial

The generated XAML part and your code-behind part share one class.

Scope

The region where a name is available. A local variable stays in its method; a field can be used by several methods.

Interface reference



This illustration is rendered from the starter XAML. Match the visible controls to x:Name and Click before adding behavior.

CHAPTER 3 / CODE TO ADD

Read the connection

WHERE: MainWindow.xaml; example only, do not add a duplicate button.

Use Copy code in the HTML tutorial for exact indentation and complete lines.

```
<Button Content="Load media" Click="Load_Click"/>
```

Read the code

Content: The user sees this label.

Click: Names the method that receives the button event.

RUN AND CHECK

Point to one visible button, its XAML Click value, and its C# handler.

Build and understand the interface

The raw shell gives you named controls and comments. Adjust the layout for your user, then connect the actions in the later chapters.

1. Read the complete raw XAML below. If starting from a blank project, replace MainWindow.xaml with it. If using the downloaded starter, it is already there; edit a few labels to fit your context.

2. Grid rows and columns divide the window. Auto uses the size of its content; * shares the remaining space. StackPanel places controls one after another; WrapPanel moves them to another line when needed.

3. Keep all x:Name and handler names while changing appearance. C# refers to those names. Keep text labels even when you add images to buttons.

4. Run the shell. Change the title or subtitle, run again, and save a screenshot. This is a layout checkpoint before behavior.

Understand the concepts

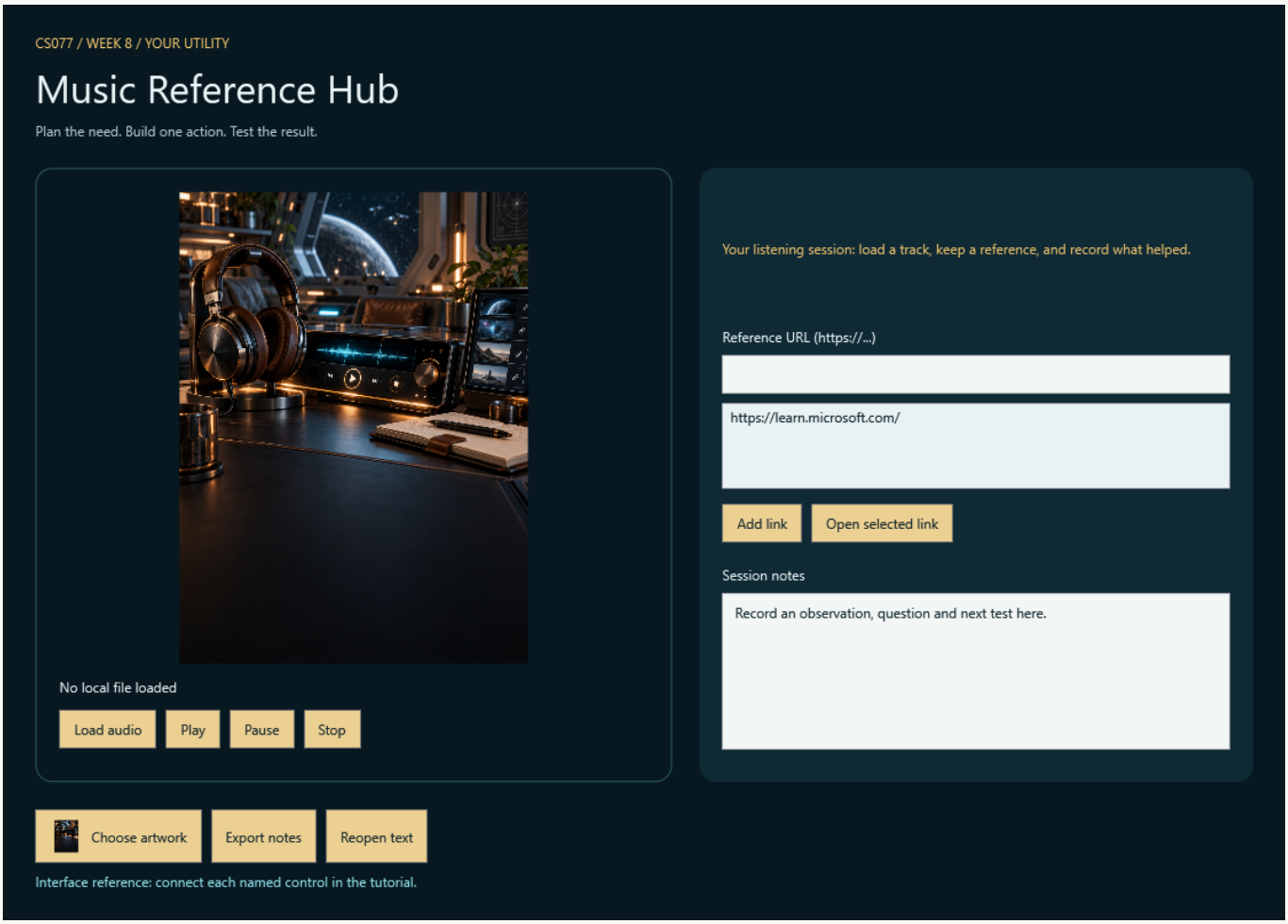
Property

A setting on an object, such as Text, Margin, Width, or Background.

Event

A notification that something happened, such as a click, timer tick, or media completion.

Interface reference



This illustration is rendered from the starter XAML. Match the visible controls to x:Name and Click before adding behavior.

CHAPTER 4 / CODE TO ADD

Raw interface

WHERE: Replace MainWindow.xaml only if starting from a blank project. Keep the matching namespace.

Use Copy code in the HTML tutorial for exact indentation and complete lines.

```
<Window x:Class="MusicReferenceHub.MainWindow"
 xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
 xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
 Title="Music Reference Hub" Width="1120" Height="850" MinWidth="900" MinHeight="780"
 Background="#071820" Foreground="#EDF8FA">
 <!-- XAML builds the visible window. C# adds the actions. Keep x:Class matched to the
    namespace. -->
 <Window.Resources>
  <Style TargetType="Button"><Setter Property="Background" Value="#EBD092"/><Setter
    Property="Foreground" Value="#10232C"/><Setter Property="Padding"
    Value="12,8"/><Setter Property="Margin" Value="0,0,8,8"/></Style>
  <Style TargetType="TextBox"><Setter Property="Padding" Value="8"/><Setter
    Property="Background" Value="#F2F7F5"/><Setter Property="Foreground"
    Value="#10232C"/></Style>
  <Style TargetType="ListBox"><Setter Property="Background" Value="#E9F2F3"/><Setter
    Property="Foreground" Value="#10232C"/></Style>
 </Window.Resources>
```

```
<Grid Margin="28"><Grid.RowDefinitions><RowDefinition Height="Auto"/><RowDefinition
   Height="*"/><RowDefinition Height="Auto"/></Grid.RowDefinitions>
 <StackPanel><TextBlock Text="CS077 / WEEK 8 / YOUR UTILITY"
   Foreground="#F1C66F"/><TextBlock Text="Music Reference Hub" FontSize="34"
   Margin="0,8"/>
  <!-- TODO: add your user and story context to this subtitle. -->
  <TextBlock Text="Plan the need. Build one action. Test the result."
   Foreground="#B5CED5"/></StackPanel>
 <Grid Grid.Row="1" Margin="0,24"><Grid.ColumnDefinitions><ColumnDefinition
   Width="1.15*"/><ColumnDefinition Width="24"/><ColumnDefinition
   Width="1*"/></Grid.ColumnDefinitions>
  <Border Padding="20" BorderBrush="#315D69" BorderThickness="1"
   CornerRadius="14"><Grid><Grid.RowDefinitions><RowDefinition
   Height="*"/><RowDefinition Height="Auto"/><RowDefinition
   Height="Auto"/></Grid.RowDefinitions>
   <Image x:Name="imgArtwork" Stretch="Uniform"/>
   <!-- VISUAL HAT: this player opens a local file. Add behavior in C#. -->
   <MediaElement x:Name="mediaPlayer" Width="1" Height="1" Opacity="0"
   IsHitTestVisible="False" LoadedBehavior="Manual" UnloadedBehavior="Stop"
   Volume="0.35" Stretch="Uniform" MediaOpened="Media_Opened"
   MediaFailed="Media_Failed" MediaEnded="Media_Ended"/>
   <TextBlock x:Name="txtNowPlaying" Grid.Row="1" Text="No local file loaded"
   Margin="0,12" TextWrapping="Wrap"/>
   <WrapPanel Grid.Row="2"><Button Content="Load audio" Click="Load_Click"/><Button
   Content="Play" Click="Play_Click"/><Button Content="Pause"
   Click="Pause_Click"/><Button Content="Stop"
   Click="Stop_Click"/></WrapPanel></Grid></Border>
  <Border Grid.Column="2" Padding="20" Background="#102A35"
   CornerRadius="14"><Grid><Grid.RowDefinitions><RowDefinition
   Height="100"/><RowDefinition Height="*"/></Grid.RowDefinitions>
   <!-- TODO: load your PNG/JPG into these named images in the Artwork chapter. -->
   <TextBlock Text="Your listening session: load a track, keep a reference, and
   record what helped." TextWrapping="Wrap" Foreground="#F1C66F"
   VerticalAlignment="Center"/>
   <Grid Grid.Row="1" Margin="0,18,0,0"><Grid><Grid.RowDefinitions><RowDefinition
   Height="Auto"/><RowDefinition Height="Auto"/><RowDefinition
   Height="75"/><RowDefinition Height="Auto"/><RowDefinition
   Height="Auto"/><RowDefinition Height="*"/></Grid.RowDefinitions>
   <TextBlock Text="Reference URL (https://...)"/><TextBox x:Name="txtLink"
   Grid.Row="1" Margin="0,8"/><ListBox x:Name="lstLinks" Grid.Row="2"/>
   <WrapPanel Grid.Row="3" Margin="0,12"><Button Content="Add link"
   Click="AddLink_Click"/><Button Content="Open selected link"
   Click="OpenLink_Click"/></WrapPanel>
   <TextBlock Grid.Row="4" Text="Session notes"/><TextBox x:Name="txtNotes"
   Grid.Row="5" MinHeight="40" AcceptsReturn="True" TextWrapping="Wrap"
   VerticalScrollBarVisibility="Auto" Margin="0,8"/></Grid></Grid>
  </Grid></Border>
 </Grid>
 <StackPanel Grid.Row="2"><WrapPanel>
```

Raw interface

WHERE: Replace MainWindow.xaml only if starting from a blank project. Keep the matching namespace.

Use Copy code in the HTML tutorial for exact indentation and complete lines.

```
   <!-- An image inside a button does not change how Click works. -->
   <Button Click="Image_Click"><StackPanel Orientation="Horizontal"><Image
    x:Name="imgButton" Width="28" Height="28" Margin="0,0,8,0"/><TextBlock
    Text="Choose artwork" VerticalAlignment="Center"/></StackPanel></Button>
   <Button Content="Export notes" Click="Save_Click"/><Button Content="Reopen text"
    Click="Open_Click"/>
  </WrapPanel><TextBlock x:Name="txtStatus" Text="Raw starter: follow the tutorial to
    add each behavior." Foreground="#95E6EF" TextWrapping="Wrap"/></StackPanel>
 </Grid>
</Window>
```

Read the code

Grid.Row: Chooses which row contains the control.

x:Name: Creates the reference used by your C# methods.

Margin: Leaves space outside the control.

Style: Gives several controls consistent settings.

RUN AND CHECK

The whole window is visible at its minimum size; all labels are readable.

Create your first GitHub checkpoint

Git records versions on your computer. GitHub keeps a remote copy that you can share. Saving a file, committing, and pushing are separate actions.

1. In Visual Studio select Git -> Create Git Repository. If the folder already belongs to a course repository, use it; do not make a nested repository.

2. Choose GitHub, sign in with the account prepared in class, and select the project folder. Use the repository access directed by the instructor. Choose Create and Push.

3. Open View -> Git Changes. Review the file list. Use the Visual Studio .gitignore below to keep generated folders out. Keep .csproj, .xaml, .cs, small Assets files, and Evidence.

4. For later changes, press Ctrl+S, enter a clear commit message in Git Changes, Commit All, then Push. Commit means a local checkpoint; Push sends that checkpoint to GitHub.

5. Open the GitHub repository in the browser. Confirm the source files appear. Share the link with a peer and test access. Keep raw starter and first behavior commits separate.

Understand the concepts

Repository

The project folder plus Git history.

Commit

A saved version with a message explaining the change.

Push

Send local commits to the connected remote repository.

.gitignore

A list of files Git should leave out. It does not remove files already tracked.

Create .gitignore

WHERE: Right-click the project -> Add -> New Item -> Text File. Name it .gitignore at the repository root.

Use Copy code in the HTML tutorial for exact indentation and complete lines.

```
.vs/
[Bb]in/
[Oo]bj/
*.user
*.suo
```

Read the code

bin and obj: Build output that Visual Studio recreates.

Assets: Small approved assets belong in the repository; do not ignore the whole folder.

RUN AND CHECK

GitHub shows the raw source and no bin, obj, or .vs folder.

Give the window data it can remember

A variable stores a value under a name. Types describe the values you can store. A collection keeps several related values.

1. Add the fields below directly inside MainWindow, above its constructor. They are not inside a button handler.

2. Replace only the existing constructor with the one below. Keep InitializeComponent() first. ItemsSource connects the visible list to the collection.

3. ObservableCollection tells WPF when items are added or removed. The list updates without rebuilding the window.

4. mediaReady and isPlaying are Boolean fields. They describe whether a file can play and whether Play was pressed. They are not audio files.

Understand the concepts

string

A text value, such as a URL, task label, or note.

int

A whole number, such as the remaining number of seconds.

bool

true or false: useful for choosing a branch.

Collection

A container holding several values. This list stores strings.

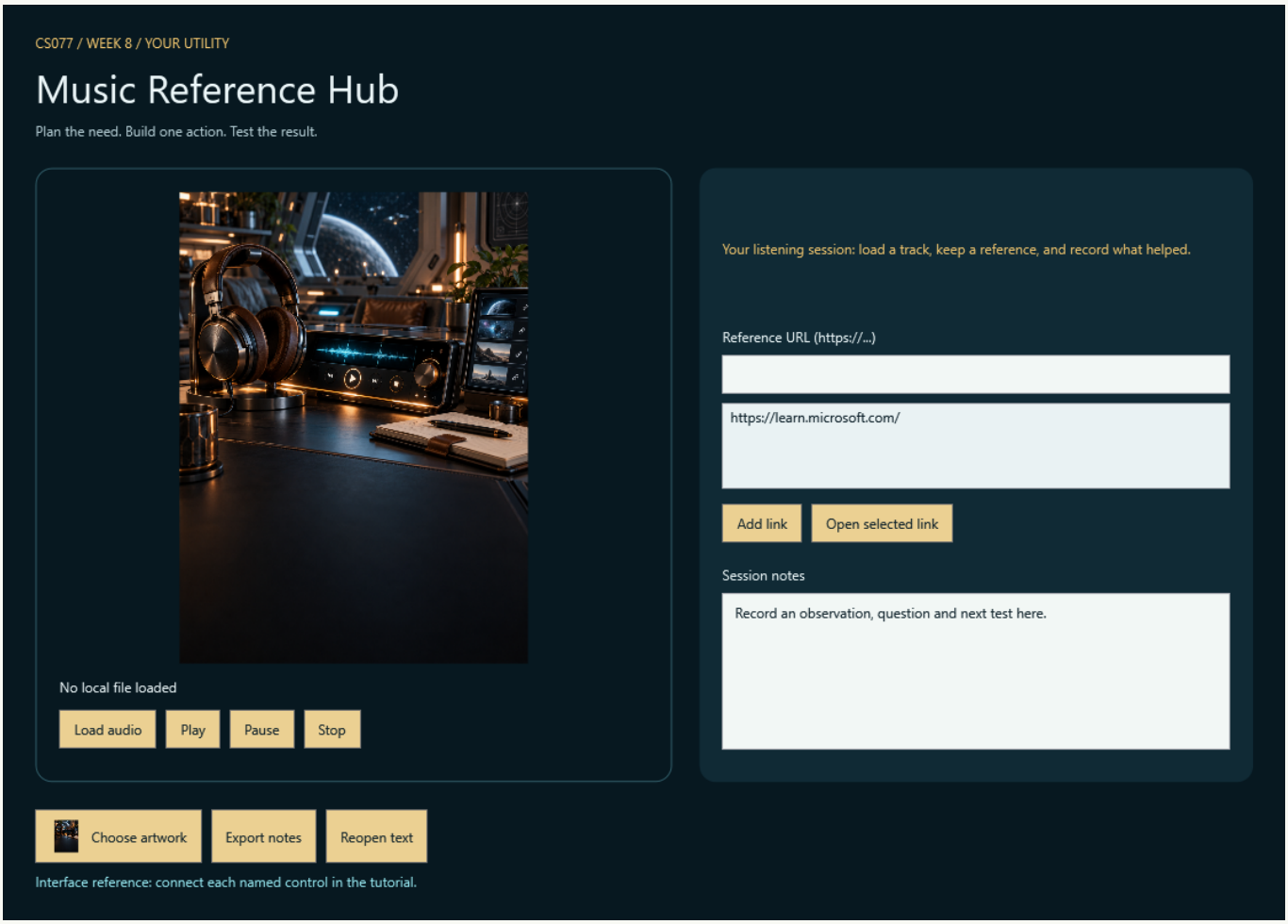
Array vs collection

An array has a fixed length after creation, such as three categories. ObservableCollection lets the user add or remove values and informs the interface.

Field vs local

Fields last as long as the window; entry in AddItem exists only while that method runs.

Interface reference



This illustration is rendered from the starter XAML. Match the visible controls to x:Name and Click before adding behavior.

CHAPTER 6 / CODE TO ADD

Add fields

WHERE: Inside MainWindow, outside all methods.

Use Copy code in the HTML tutorial for exact indentation and complete lines.

```
    // Fields stay available to methods throughout this window's lifetime.
    private readonly System.Collections.ObjectModel.ObservableCollection<string> links
    = new();
    private bool mediaReady = false;
    private bool isPlaying = false;
```

Read the code

readonly: The field keeps the same collection or timer object; its contents or state can still change.

new(): Creates an object of the declared type.

CHAPTER 6 / CODE TO ADD

Replace constructor

WHERE: Replace public MainWindow() and its braces. Do not add a second constructor.

Use Copy code in the HTML tutorial for exact indentation and complete lines.

```
    public MainWindow()
    {
        InitializeComponent();
        lstLinks.ItemsSource = links;
    }
```

Read the code

InitializeComponent: Builds named controls before the next line uses them.

ItemsSource: The control reads the collection as its list.

RUN AND CHECK

Build succeeds. The list is empty until you implement the next chapter.

Load and play your music

A local media file goes into MediaElement.Source. The player opens it, reports readiness, and responds to button events.

1. Use the Music shelf below to preview and download any course track. Or choose another instructor-approved audio file. Save it in Assets/Media.

2. Add LoadLocalMedia as a new helper method inside MainWindow. Replace the existing Load_Click, Play_Click, Pause_Click, Stop_Click, Media_Opened, Media_Failed, and Media_Ended stubs with the definitions below.

3. LoadedBehavior="Manual" in XAML lets your methods control playback. Source receives a Uri, which identifies a local file. Path.GetFileName shows just its filename.

4. Media opens asynchronously: setting Source does not guarantee it is ready. MediaOpened enables playback; MediaFailed supplies a recovery message.

5. A YouTube address is a web reference. Add it in the Links chapter so it opens in the browser. Do not place a watch-page address into the local media player.

Understand the concepts

Uri

An object representing a location, including an absolute path to a local file.

Condition

An expression that decides which branch runs. !mediaReady means “not ready.”

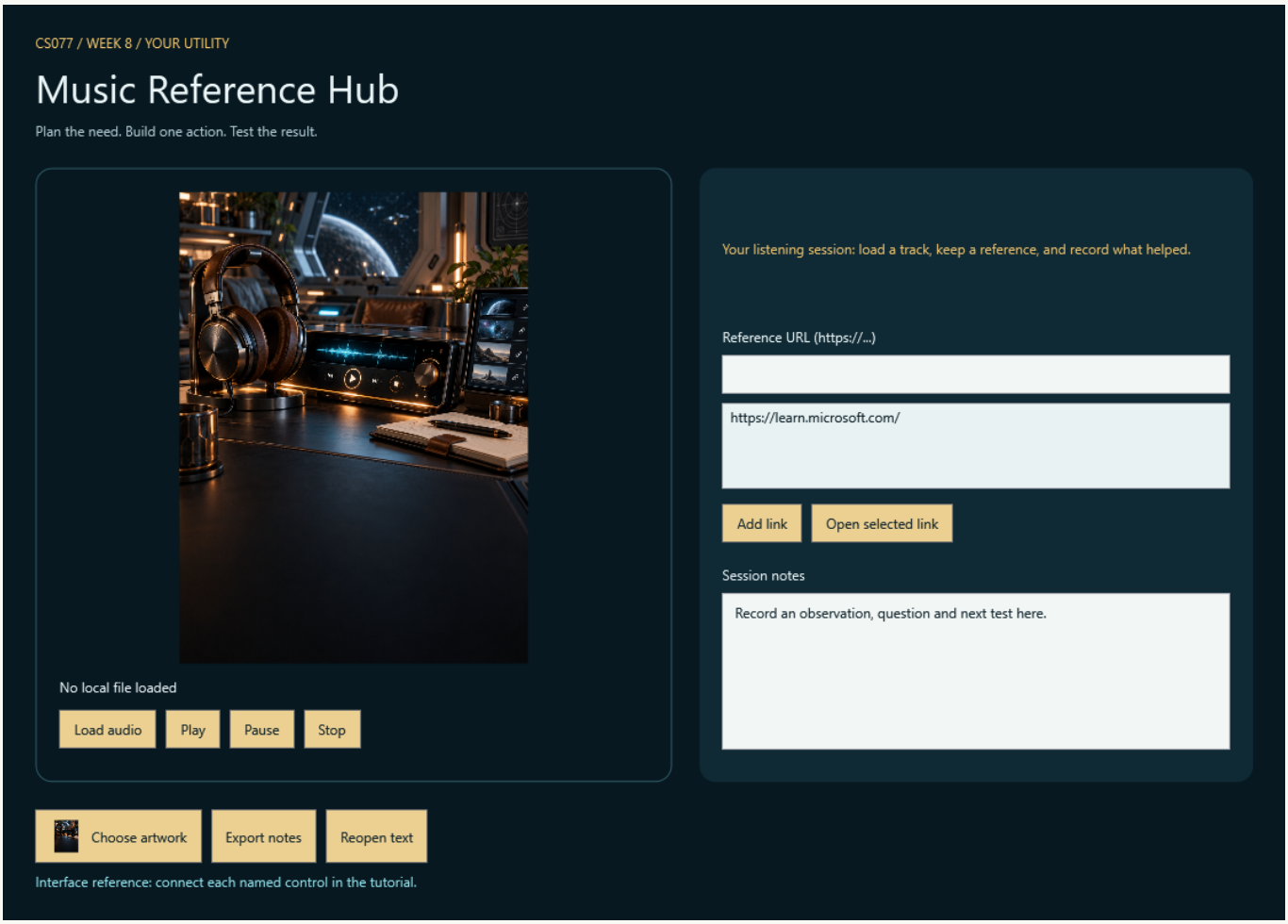
Early return

Stops the current method when a condition requires another user action.

Async opening

The player finishes loading later and raises an event when it is ready.

Interface reference



This illustration is rendered from the starter XAML. Match the visible controls to x:Name and Click before adding behavior.

Choose any course track

Open the Music HTML tutorial to preview and download Voidrunner, The Foundry, The Relay, or Star Map. Save a track in Assets/Media; Load audio chooses the local file. You may also use another instructor-approved local file.

CHAPTER 7 / CODE TO ADD

Load and playback methods

WHERE: Inside MainWindow. Replace the seven named placeholder handlers; add the helper once.

Use Copy code in the HTML tutorial for exact indentation and complete lines.

```
    private void LoadLocalMedia()
    {
        var dialog = new OpenFileDialog
        { Filter = "Audio|*.mp3;*.wav;*.wma|All files|*.*" };
        if (dialog.ShowDialog() != true)
        {
            txtStatus.Text = "Load canceled. Choose a file when ready.";
            return;
        }
        mediaPlayer.Stop();
        isPlaying = false;
```

```
        mediaReady = false;
        mediaPlayer.Source = new Uri(dialog.FileName);
        txtNowPlaying.Text = Path.GetFileName(dialog.FileName);
        txtStatus.Text = "Opening media. Wait for Ready before Play.";
    }

    private void Media_Opened(object sender, RoutedEventArgs e)
    {
        mediaReady = true;
        txtStatus.Text = "Ready. Press Play.";
    }

    private void Media_Failed(object sender, ExceptionRoutedEventArgs e)
    {
        mediaReady = false;
        isPlaying = false;
        txtStatus.Text = "Media could not open. Try another local file or format.";
    }

    private void Media_Ended(object sender, RoutedEventArgs e)
    {
```

Load and playback methods

WHERE: Inside MainWindow. Replace the seven named placeholder handlers; add the helper once.

Use Copy code in the HTML tutorial for exact indentation and complete lines.

```
        mediaPlayer.Stop();
        isPlaying = false;
        txtStatus.Text = "Playback finished. Press Play to restart.";
    }

    private void Play_Click(object sender, RoutedEventArgs e)
    {
        if (!mediaReady) { txtStatus.Text = "Load a supported local file first.";
    return; }
        mediaPlayer.Play();
        isPlaying = true;
        txtStatus.Text = "Playing.";
    }

    private void Pause_Click(object sender, RoutedEventArgs e)
    {
        if (!mediaReady) { txtStatus.Text = "Load a file first."; return; }
        if (!isPlaying) { txtStatus.Text = "Playback is already paused or stopped.";
    return; }
        mediaPlayer.Pause();
        isPlaying = false;
        txtStatus.Text = "Paused. Play continues from this point.";
    }

    private void Stop_Click(object sender, RoutedEventArgs e)
    {
        if (!mediaReady) { txtStatus.Text = "Load a file first."; return; }
        mediaPlayer.Stop();
        isPlaying = false;
        txtStatus.Text = "Stopped. Play starts from the beginning.";
    }

    private void Load_Click(object sender, RoutedEventArgs e) => LoadLocalMedia();
```

Read the code

ShowDialog() == true: The user chose a file; cancel leaves the current project intact.

Source = new Uri(...): Tells the player where the file is.

mediaReady: Prevents Play before a file has opened.

Play / Pause / Stop: Start or resume; pause at the current point; stop and reset to the beginning.

RUN AND CHECK

Cancel load, press Play with no file, load a supported file, then test Play, Pause, Stop and replay. Record a screenshot and push the 8A checkpoint.

8B / ADVANCE / CHAPTER 8

Keep and open useful reference links

The link list stores URLs as text. A separate method opens the selected valid address in the default browser.

1. Add AddLink and OpenSelectedLink inside MainWindow. Replace AddLink_Click and OpenLink_Click with the small handlers below.

2. Uri.TryCreate checks the shape of the address. The scheme check allows http and https. A path or arbitrary text is not accepted as a web reference.

3. UseShellExecute asks Windows to use the default browser. The try/catch gives feedback if Windows cannot open it.

4. Test a blank entry, random text, a complete https URL, and opening without selecting a link. Use course or research references that support your context.

Understand the concepts

&& / ||

AND needs both expressions; OR needs at least one. Here the invalid-address conditions use OR.

out

TryCreate places its parsed Uri in the variable named uri.

try/catch

Run code that may fail; handle the exception with a useful recovery message.

CHAPTER 8 / CODE TO ADD

Links and browser methods

WHERE: Add the two helpers; replace AddLink_Click and OpenLink_Click.

Use Copy code in the HTML tutorial for exact indentation and complete lines.

```
    private void AddLink()
    {
        string address = txtLink.Text.Trim();
        if (!Uri.TryCreate(address, UriKind.Absolute, out Uri? uri)
            || (uri.Scheme != "https" && uri.Scheme != "http"))
        {
            txtStatus.Text = "Enter a complete https:// or http:// link.";
            return;
        }
        links.Add(uri.AbsoluteUri);
        txtLink.Clear();
        txtStatus.Text = "Reference added. Select it, then Open link.";
    }

    private void OpenSelectedLink()
```

```
    {
        if (lstLinks.SelectedItem is not string address)
        {
            txtStatus.Text = "Select a reference link first.";
            return;
        }
        try
        {
            System.Diagnostics.Process.Start(new
    System.Diagnostics.ProcessStartInfo(address)
            { UseShellExecute = true });
            txtStatus.Text = "Reference opened in your browser.";
        }
        catch (Exception ex)
        {
            txtStatus.Text = "Could not open the browser: " + ex.Message;
        }
    }
```

Links and browser methods

WHERE: Add the two helpers; replace AddLink_Click and OpenLink_Click.

Use Copy code in the HTML tutorial for exact indentation and complete lines.

```
    private void AddLink_Click(object sender, RoutedEventArgs e) => AddLink();
    private void OpenLink_Click(object sender, RoutedEventArgs e) =>
    OpenSelectedLink();
```

Read the code

links.Add: Stores the validated address in the collection.

SelectedItem: The entry the user chose in the ListBox.

Process.Start: Asks the operating system to open the browser.

RUN AND CHECK

Invalid entries are rejected; a selected http or https link opens in the browser.

Export and reopen a readable record

A method parameter receives the text you want to save. File.WriteAllText writes it; File.ReadAllText retrieves it.

1. Add SaveText and replace Open_Click with the definitions below. SaveText takes one string parameter and handles cancellation and errors.

2. Replace Save_Click with the one-line handler. It passes txtNotes.Text into SaveText.

3. Export to a writable folder. Open the file in Notepad and confirm its content. Press Reopen text to bring it back into the app.

4. This chapter saves notes only. It does not restore links or media choices. Mention that limit in README; structured persistence is later work.

5. Never overwrite the only copy of useful work during a test. Use a new test filename and confirm cancel leaves your text in the window.

Understand the concepts

Parameter

string text is the value SaveText receives from its caller.

void

This method does work without returning a value.

Return type

BuildReport returns a string; its caller can pass that result to another method.

foreach loop

Visit each value in a collection and perform the body for it.

Text file methods

WHERE: Add SaveText; replace Open_Click. using System.IO and Microsoft.Win32 belong at the top.

Use Copy code in the HTML tutorial for exact indentation and complete lines.

```
    private void SaveText(string text)
    {
        var dialog = new SaveFileDialog { Filter = "Text files|*.txt", FileName =
    "music-record.txt" };
        if (dialog.ShowDialog() != true)
        {
            txtStatus.Text = "Save canceled. Your text stays here.";
            return;
        }
```

```
        try
        {
            File.WriteAllText(dialog.FileName, text);
            txtStatus.Text = "Saved: " + Path.GetFileName(dialog.FileName);
        }
        catch (Exception ex)
        {
            txtStatus.Text = "Could not save. Choose a writable folder: " + ex.Message;
        }
    }

    private void Open_Click(object sender, RoutedEventArgs e)
    {
        var dialog = new OpenFileDialog { Filter = "Text files|*.txt" };
        if (dialog.ShowDialog() != true)
        {
            txtStatus.Text = "Open canceled. Your current text stays here.";
            return;
        }
        try
        {
            txtNotes.Text = File.ReadAllText(dialog.FileName);
            txtStatus.Text = "Text reopened. Review it before editing.";
        }
```

CHAPTER 9 / CODE CONTINUED

Text file methods

WHERE: Add SaveText; replace Open_Click. using System.IO and Microsoft.Win32 belong at the top.

Use Copy code in the HTML tutorial for exact indentation and complete lines.

```
        catch (Exception ex)
        {
            txtStatus.Text = "Could not read the file: " + ex.Message;
        }
    }
```

Read the code

WriteAllText: Writes the complete text to the chosen path.

ReadAllText: Reads the complete text into the document or notes box.

catch: Reports the error so the user can choose another file or folder.

CHAPTER 9 / CODE TO ADD

Export handler

WHERE: Replace Save_Click.

Use Copy code in the HTML tutorial for exact indentation and complete lines.

```
    private void Save_Click(object sender, RoutedEventArgs e) =>
    SaveText(txtNotes.Text);
```

Read the code

Argument: The actual value passed when calling SaveText.

Method composition: A click delegates work to methods with one job each.

RUN AND CHECK

Write a note or contract, export, inspect in Notepad, reopen, cancel, and recover from an unsuitable file.

Add artwork and an image button

Images should make the user's purpose recognizable while labels explain the action.

1. Download the course image or choose an approved PNG/JPG. Save it in Assets/Images. Keep a small image-sources.txt in Evidence with its source and permission or generation note.

2. In Visual Studio, right-click Assets -> Add -> Existing Item. Select the image. For a fixed XAML image, set its Build Action to Resource; then use Source="Assets/Images/your-file.png".

3. For a user-chosen image, replace Image_Click with the code below. It loads a BitmapImage into both the artwork panel and the small image inside Choose artwork.

4. Read the button XAML: the Image and TextBlock are children of a StackPanel, which is the button's content. The same Click handler still runs.

5. Test a narrow window, keyboard Tab and Enter, and readable labels. Ask a peer what the image suggests before explaining it.

Understand the concepts

BitmapImage

An object containing pixels WPF can show in an Image control.

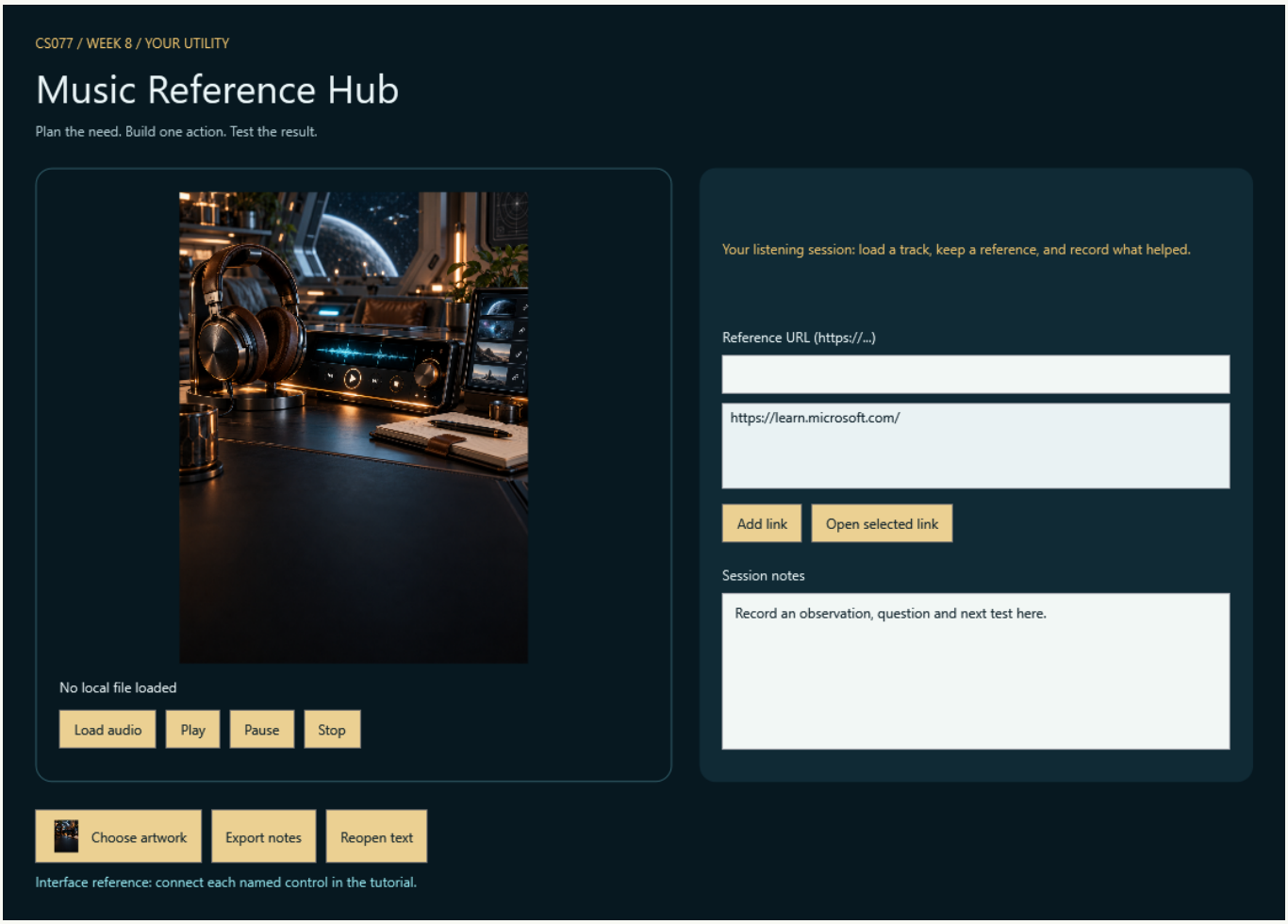
Resource

A project asset embedded for use by the app.

Runtime file

A file the user chooses while the app runs; its path can differ on another machine.

Interface reference



This illustration is rendered from the starter XAML. Match the visible controls to x:Name and Click before adding behavior.

CHAPTER 10 / CODE TO ADD

Artwork loader

WHERE: Replace Image_Click; all named images already exist in the raw XAML.

Use Copy code in the HTML tutorial for exact indentation and complete lines.

```
    private void Image_Click(object sender, RoutedEventArgs e)
    {
        var dialog = new OpenFileDialog { Filter = "Images|*.png;*.jpg;*.jpeg" };
        if (dialog.ShowDialog() != true)
        {
            txtStatus.Text = "Image choice canceled. Your work stays here.";
            return;
        }
        try
        {
            // Load the image fully so its original file is not left locked.
            var bitmap = new System.Windows.Media.Imaging.BitmapImage();
            bitmap.BeginInit();
            bitmap.CacheOption = System.Windows.Media.Imaging.BitmapCacheOption.OnLoad;
            bitmap.UriSource = new Uri(dialog.FileName);
            bitmap.EndInit();
            imgArtwork.Source = bitmap;
```

```
            imgButton.Source = bitmap;
            txtStatus.Text = "Artwork loaded. Check labels and contrast.";
        }
        catch (Exception ex)
        {
            txtStatus.Text = "Choose a valid PNG or JPG: " + ex.Message;
        }
    }
```

Read the code

BeginInit / EndInit: Configure the image before WPF uses it.

OnLoad: Reads the pixels now so the original image can be released.

Source: The image object displayed by each Image control.

CHAPTER 10 / CODE TO ADD

Read the image button

WHERE: MainWindow.xaml; already in the raw starter.

Use Copy code in the HTML tutorial for exact indentation and complete lines.

```
<Button Click="Image_Click">
 <StackPanel Orientation="Horizontal">
  <Image x:Name="imgButton" Width="28" Height="28"/>
  <TextBlock Text="Choose artwork"/>
 </StackPanel>
</Button>
```

Read the code

Button: Receives the click.

StackPanel: Arranges the image and label.

RUN AND CHECK

Choose a valid image, cancel, try another file, and confirm text labels still explain the buttons.

Test, ask AI, and resolve a real unknown

Record what happened. A working happy path alone does not prove the user can recover.

1. Test: Play before load; cancel file selection; supported media playback; Pause; Stop; replay; invalid media; blank or invalid URL; no selected link; valid browser link; export; reopen; image cancel.

2. Write TestNotes.md with input, predicted result, observed result, and next test. Reuse the 8A/8B whiteboard and Discord peer checks; keep screenshots and lecture notes.

3. Use the required focused AI prompt below. Ask for one explanation or change, paste only what you can place and test, then record kept, changed, or rejected.

4. Check the final code reference only after following the chapters. Compare names and braces to recover from a compile error. Do not paste a second whole class into the same file.

Understand the concepts

Test case

One specific input or action with an expected result.

Debugging

Use evidence to locate and change the cause of an error.

Required AI question

I am building Music Reference Hub in C# WPF. My unknown is [one question]. Explain the smallest useful change, the exact file and method, and one test. Explain the variables and method calls. I will record what I kept, changed or rejected.

RUN AND CHECK

Your test record includes normal operation, invalid input, cancellation and recovery. Commit and push the accepted 8B checkpoint.

Package the app and submit the GitHub link

Two class checkpoints supply the weekly work. The final pass makes the app and its evidence understandable to someone else.

1. Complete README.md: user, purpose, required Windows/.NET version, how to open/run, controls, source notes, known limits, and test results. Add PATTERN.md explaining one reusable method, event, validation, UI, file or media pattern.

2. Add Context.md, assumptions, prototype, whiteboard photos, concise Discord records, AI test, screenshots, lecture notes and downloaded receipt. Both Action Records must be accepted before weekly grading under the course rule.

3. Commit in Visual Studio and Push. Open GitHub: inspect the source and evidence. Ask a peer to test the link using the instructor-directed access.

4. Large music and video files should not bloat Git. GitHub blocks ordinary files over 100 MB. For large files, keep a course-authorized download link and setup instructions or use the instructor-approved storage route. Commit small artwork and your code.

5. Submit the tested GitHub repository URL through the Week 8 Canvas assignment before the next class, according to Canvas. Include the receipt in the repository. Verify the requirements before submitting.

Understand the concepts

Reusable pattern

A small piece of structure or behavior you can explain and adapt to another app.

Release

The version you have checked, documented, and handed to another user.

RUN AND CHECK

A peer can find instructions, open the project, locate evidence and use your required workflow.

Complete code reference

Use this to compare structure after completing the steps. Read and test each chapter before consulting the whole file.

1. If your class has duplicate handlers or mismatched braces, back up the file, compare it with the reference, and explain the corrected placement in your notes.

2. This reference assumes the raw XAML above, its control names, and the original namespace.

CHAPTER 13 / CODE TO ADD

Full code-behind

WHERE: Replace MainWindow.xaml.cs only for recovery after the steps; never paste it inside an existing class.

Use Copy code in the HTML tutorial for exact indentation and complete lines.

```
using System;
using System.IO;
using System.Windows;
using Microsoft.Win32;

namespace MusicReferenceHub;

// MainWindow is a class. Put fields and methods inside these outer braces.
public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
        lstLinks.ItemsSource = links;
    }

    // Fields stay available to methods throughout this window's lifetime.
    private readonly System.Collections.ObjectModel.ObservableCollection<string> links
    = new();
    private bool mediaReady = false;
    private bool isPlaying = false;

    private void LoadLocalMedia()
    {
        var dialog = new OpenFileDialog
        { Filter = "Audio|*.mp3;*.wav;*.wma|All files|*.*" };
        if (dialog.ShowDialog() != true)
        {
            txtStatus.Text = "Load canceled. Choose a file when ready.";
            return;
        }
        mediaPlayer.Stop();
        isPlaying = false;
```

Full code-behind

WHERE: Replace MainWindow.xaml.cs only for recovery after the steps; never paste it inside an existing class.

Use Copy code in the HTML tutorial for exact indentation and complete lines.

```
        mediaReady = false;
        mediaPlayer.Source = new Uri(dialog.FileName);
        txtNowPlaying.Text = Path.GetFileName(dialog.FileName);
        txtStatus.Text = "Opening media. Wait for Ready before Play.";
    }

    private void Media_Opened(object sender, RoutedEventArgs e)
    {
        mediaReady = true;
        txtStatus.Text = "Ready. Press Play.";
    }

    private void Media_Failed(object sender, ExceptionRoutedEventArgs e)
    {
        mediaReady = false;
        isPlaying = false;
        txtStatus.Text = "Media could not open. Try another local file or format.";
    }

    private void Media_Ended(object sender, RoutedEventArgs e)
    {
        mediaPlayer.Stop();
        isPlaying = false;
        txtStatus.Text = "Playback finished. Press Play to restart.";
    }

    private void Play_Click(object sender, RoutedEventArgs e)
    {
        if (!mediaReady) { txtStatus.Text = "Load a supported local file first.";
    return; }
        mediaPlayer.Play();
        isPlaying = true;
        txtStatus.Text = "Playing.";
```

Full code-behind

WHERE: Replace MainWindow.xaml.cs only for recovery after the steps; never paste it inside an existing class.

Use Copy code in the HTML tutorial for exact indentation and complete lines.

```
    }

    private void Pause_Click(object sender, RoutedEventArgs e)
    {
        if (!mediaReady) { txtStatus.Text = "Load a file first."; return; }
        if (!isPlaying) { txtStatus.Text = "Playback is already paused or stopped.";
    return; }
        mediaPlayer.Pause();
        isPlaying = false;
        txtStatus.Text = "Paused. Play continues from this point.";
    }

    private void Stop_Click(object sender, RoutedEventArgs e)
    {
        if (!mediaReady) { txtStatus.Text = "Load a file first."; return; }
        mediaPlayer.Stop();
        isPlaying = false;
        txtStatus.Text = "Stopped. Play starts from the beginning.";
    }

    private void Load_Click(object sender, RoutedEventArgs e) => LoadLocalMedia();

    private void AddLink()
    {
        string address = txtLink.Text.Trim();
        if (!Uri.TryCreate(address, UriKind.Absolute, out Uri? uri)
            || (uri.Scheme != "https" && uri.Scheme != "http"))
        {
            txtStatus.Text = "Enter a complete https:// or http:// link.";
            return;
        }
        links.Add(uri.AbsoluteUri);
        txtLink.Clear();
```

Full code-behind

WHERE: Replace MainWindow.xaml.cs only for recovery after the steps; never paste it inside an existing class.

Use Copy code in the HTML tutorial for exact indentation and complete lines.

```
        txtStatus.Text = "Reference added. Select it, then Open link.";
    }

    private void OpenSelectedLink()
    {
        if (lstLinks.SelectedItem is not string address)
        {
            txtStatus.Text = "Select a reference link first.";
            return;
        }
        try
        {
            System.Diagnostics.Process.Start(new
System.Diagnostics.ProcessStartInfo(address)
            { UseShellExecute = true });
            txtStatus.Text = "Reference opened in your browser.";
        }
        catch (Exception ex)
        {
            txtStatus.Text = "Could not open the browser: " + ex.Message;
        }
    }

    private void AddLink_Click(object sender, RoutedEventArgs e) => AddLink();
    private void OpenLink_Click(object sender, RoutedEventArgs e) =>
    OpenSelectedLink();

    private void SaveText(string text)
    {
        var dialog = new SaveFileDialog { Filter = "Text files|*.txt", FileName =
    "music-record.txt" };
        if (dialog.ShowDialog() != true)
        {
            txtStatus.Text = "Save canceled. Your text stays here.";
            return;
```

Full code-behind

WHERE: Replace MainWindow.xaml.cs only for recovery after the steps; never paste it inside an existing class.

Use Copy code in the HTML tutorial for exact indentation and complete lines.

```
        }
        try
        {
            File.WriteAllText(dialog.FileName, text);
            txtStatus.Text = "Saved: " + Path.GetFileName(dialog.FileName);
        }
        catch (Exception ex)
        {
            txtStatus.Text = "Could not save. Choose a writable folder: " + ex.Message;
        }
    }

    private void Open_Click(object sender, RoutedEventArgs e)
    {
        var dialog = new OpenFileDialog { Filter = "Text files|*.txt" };
        if (dialog.ShowDialog() != true)
        {
            txtStatus.Text = "Open canceled. Your current text stays here.";
            return;
        }
        try
        {
            txtNotes.Text = File.ReadAllText(dialog.FileName);
            txtStatus.Text = "Text reopened. Review it before editing.";
        }
        catch (Exception ex)
        {
            txtStatus.Text = "Could not read the file: " + ex.Message;
        }
    }

    private void Save_Click(object sender, RoutedEventArgs e) =>
    SaveText(txtNotes.Text);
```

Full code-behind

WHERE: Replace MainWindow.xaml.cs only for recovery after the steps; never paste it inside an existing class.

Use Copy code in the HTML tutorial for exact indentation and complete lines.

```
    private void Image_Click(object sender, RoutedEventArgs e)
    {
        var dialog = new OpenFileDialog { Filter = "Images|*.png;*.jpg;*.jpeg" };
        if (dialog.ShowDialog() != true)
        {
            txtStatus.Text = "Image choice canceled. Your work stays here.";
            return;
        }
        try
        {
            // Load the image fully so its original file is not left locked.
            var bitmap = new System.Windows.Media.Imaging.BitmapImage();
            bitmap.BeginInit();
            bitmap.CacheOption = System.Windows.Media.Imaging.BitmapCacheOption.OnLoad;
            bitmap.UriSource = new Uri(dialog.FileName);
            bitmap.EndInit();
            imgArtwork.Source = bitmap;
            imgButton.Source = bitmap;
            txtStatus.Text = "Artwork loaded. Check labels and contrast.";
        }
        catch (Exception ex)
        {
            txtStatus.Text = "Choose a valid PNG or JPG: " + ex.Message;
        }
    }
}
```

Read the code

Class braces: Contain the constructor, fields, methods and handlers.

Methods: Use values from fields or parameters, then update controls or return data.

RUN AND CHECK

Build succeeds and your test record explains your version.